



K

You are expert backend developer who is proficient in building ms using springboot and java and postgresql db. Help me prepare me to complete this backedn work assignment . It is using java , springboot. Please m.



Let's break down this assessment so you can approach it with total clarity and confidence. This is a classic senior-level backend design challenge that tests both your coding hygiene and your architectural mindset.

## Part 1: The Assignment Explained Simply

In plain English, you need to build a mini backend ecosystem for a software-as-a-service (SaaS) e-commerce platform where multiple independent businesses (tenants) share the same application infrastructure, but **their data and operations must be completely isolated from one another**.

Here is what you are building:

1. `order-service` : The core API where merchants create, update, and cancel customer orders. Every request must know *which* tenant is making it.
2. `notification-service` : A service that reacts to order events (like order creation, updates, or completion) and simulates sending customer notifications while keeping an audit log.
3. **Infrastructure**: Both services must connect to a PostgreSQL database, run inside Docker containers, and spin up together seamlessly using a single `docker-compose up` command.

The biggest differentiator here isn't just writing working code—it's proving **how** you handle multi-tenancy across every layer (HTTP headers, security, database queries) and **how** you handle system resilience when things fail.

## Part 2: Proposed Implementation Plan

Review the step-by-step roadmap below. Take a look, let me know what you think, or if you'd like to adjust any part of the stack or design before we start writing code!

## Phase 1: Architectural Design & Domain Modeling

**Multi-Tenancy Strategy:** Use a shared database with a shared schema approach, where every table includes a `tenant_id` column. We will intercept incoming requests via a custom Spring `HandlerInterceptor`, extract a tenant header (e.g., `X-Tenant-ID`), store it in a request-scoped `TenantContext` (backed by `ThreadLocal`), and enforce it automatically in database queries.

**Order Lifecycle & Statuses:** Define a clean state machine for orders (e.g., `CREATED` → `PROCESSING` → `COMPLETED`, with a separate `CANCELLED` path subject to business rules like "cannot cancel once processing").

**Service Communication:** Establish synchronous REST communication between `order-service` and `notification-service` using Spring Cloud OpenFeign or `RestClient`, backed by **Resilience4j** (Circuit Breaker & Retry) to handle partial failures gracefully.

## Phase 2: Database Schema & Setup ( `order-service` & `notification-service` )

Design the PostgreSQL tables with proper indexing, foreign keys, and strict `tenant_id` isolation columns.

Configure Spring Data JPA repositories with query specifications or Hibernate filters to ensure zero data leakage between tenants.

## Phase 3: Building `order-service`

**Endpoints:**

`POST /api/orders` (Create order)

`PUT /api/orders/{id}` (Update order lifecycle)

`POST /api/orders/{id}/cancel` (Cancel order with validation rules)

**Validation & Error Handling:** Implement global exception handling ( `@ControllerAdvice` ) returning structured, consistent JSON error responses.

## Phase 4: Building `notification-service`

**Endpoints:**

`POST /api/notifications` (Endpoint receiving trigger events from `order-service` )

**Behavior:** Simulate sending notifications (logging structured payloads) and persist notification records in the database tagged with the correct `tenant_id` .

## Phase 5: Testing & Resilience

Write comprehensive unit tests (JUnit 5, Mockito) for business logic.

Write integration tests using Spring Boot Test and Testcontainers (spinning up a real PostgreSQL container for tests).

## Phase 6: Containerization & Documentation

Create optimized multi-stage `Dockerfile` s for both Java 21 Spring Boot applications.

Write a robust `docker-compose.yml` to spin up PostgreSQL, `order-service` , and `notification-service` together.

Write a professional, comprehensive `README.md` covering architecture, multi-tenancy decisions, assumptions, and API documentation.

do i have to use java 21 ? how they will know if i used java 17 vs java 21 to implement this by looking at my code? is it pom.xml or build.gradle? i am not suing any latest feature of java21 either

Yes, **they will check your build configuration files** ( `pom.xml` or `build.gradle` ) and your Docker setup.

Here is how they will know:

1. **The Build File:** In `pom.xml` , they will look at `<java.version>21</java.version>` (or the Maven compiler plugin configuration). In `build.gradle` , they will look at

`sourceCompatibility = JavaVersion.VERSION_21` . If it says `17` , it immediately flags that you didn't follow the explicit requirement ("Java 21 or higher").

2. **The Docker Image:** Your `Dockerfile` will likely use a base image like `eclipse-temurin:21-jdk` or `amazoncorretto:21` .

## Can you just set it to Java 21 even if you aren't using new features?

**Yes, absolutely.** Java 21 is fully backward compatible with Java 17 code. You can target Java 21 in your `pom.xml` and write standard, clean Java 17-style code without touching any Java 21-specific features (like virtual threads or pattern matching switch expressions), and it will compile and run seamlessly.

Since it takes literally 5 seconds to change the version number in your build file and `Dockerfile`, **it is strongly recommended to set it to Java 21** so you don't get penalized during automated or manual screening for missing a strict requirement.

## Ready for the Next Step?

Since you agree with the plan and we've cleared up the Java version requirement, let's start tackling the code!

Would you like to begin with **Phase 1 & 2** (defining the database schema and multi-

how this docker compose works? what if i do not have  
docker ? i could run those two microservices in two separate  
terminal correct?

Yes, you can run both microservices in two separate terminals without Docker, as long as you have a local PostgreSQL instance running on your machine.

## How Docker Compose Works Here

Docker Compose is simply an orchestration tool that reads your `docker-compose.yml` file and spins up three containers on a shared virtual network:

1. A **PostgreSQL** database container.
2. Your `order-service` container.
3. Your `notification-service` container.

It automatically configures environment variables so the services can talk to PostgreSQL and to each other using container names as hostnames (e.g., `jdbc:postgresql://postgres:5432/...`).

## How to Run Locally Without Docker

If you don't have Docker installed (or prefer not to use it for local development), you can run everything natively:

### 1. Start PostgreSQL Locally:

Install PostgreSQL on your machine and create a database (e.g., `ecommerce_db`).

Update your `application.yml` files in both services to point to `localhost:5432` with your local database credentials.

### 2. Run `order-service` in Terminal 1:

Bash



```
cd order-service
./mvnw spring-boot:run
```

*Verification:* Check the terminal logs for

Started `OrderServiceApplication` in X.XXX seconds and verify the port (e.g., 8080).

### 3. Run `notification-service` in Terminal 2:

Bash



```
cd notification-service
./mvnw spring-boot:run
```

**Verification:** Ensure it starts successfully on a different port (e.g., 8081 ) by checking the logs.

**Note for your submission:** *Even if you develop and test locally using terminals, the assessment explicitly requires a `docker-compose.yml` file and public Docker images so the reviewers can test it with a single command ( `docker compose up` ). You can write the code locally first, and then create the Dockerfiles and Compose file at the end!*

even with docker , i need to start postgres db locally right?

**No, you do not need to install or start PostgreSQL locally on your machine if you use Docker.**

That is the core benefit of using Docker Compose. The `docker-compose.yml` file will include a PostgreSQL service definition. When you run a single command in your terminal:

Bash



```
docker compose up
```

Docker will automatically:

1. Download the official PostgreSQL database image.
2. Spin up a dedicated PostgreSQL container in the background.
3. Make it available to your `order-service` and `notification-service` over a shared internal network.

## How your Spring Boot app talks to it

In your Spring Boot `application.yml` , instead of pointing to `localhost:5432` , you configure the datasource URL to use the container name as the hostname:



## YAML

```
spring:
  datasource:
    url: jdbc:postgresql://postgres:5432/ecommerce_db
    username: postgres
    password: password
```

(Here, `postgres` is the service name defined in your `docker-compose.yml`, which Docker resolves automatically).

## Ready to start building?

Since we have our plan locked in and you understand how Docker and local running work, we can start writing the actual code and configuration files.

Would you like to start with **Step 1: Setting up the `order-service` project structure, `pom.xml` (with Java 21), and the multi-tenant database configuration ( `TenantContext` & `Interceptor`)?**

i can go and check in postgres sql all the orders goign to db  
right? it will be available t o me locally?

**Yes, absolutely.** Even though PostgreSQL is running inside a Docker container, you can easily inspect, query, and manage all your tables and order records directly from your local machine.

When you configure your `docker-compose.yml`, you expose PostgreSQL's port to your host machine. Here is how you can access it:

## 1. Using a Database GUI Client (Recommended)

You can connect to the database using any standard tool like **DBeaver**, **pgAdmin**, **DataGrip**, **VS Code (Database Client extension)**, or **IntelliJ's built-in database tool**.

Use these connection details:

**Host:** `localhost`

**Port:** `5432` (or whatever port you map in Docker Compose)

**Database Name:** `ecommerce_db` (or whatever you name it)

**Username:** `postgres`

**Password:** `password`

Once connected, you can run standard SQL queries like `SELECT * FROM orders;` to see every order coming in from your Spring Boot app in real time.

## 2. Using the Command Line

If you prefer the terminal, you can open a `psql` shell directly inside the running container using Docker:

Bash



```
docker exec -it <postgres_container_name> psql -U postgres -d ecommerce_db
```

*(Once inside, you can type `\dt` to list tables or `SELECT * FROM orders;` to view records).*

## Ready to start building?

Now that you know how local development, Docker, and database inspection work, we are fully set to start writing the code.

Would you like us to generate the project files for `order-service` (starting with `pom.xml`, the database configuration, and the multi-tenant `TenantContext` interceptor)?

using what rest http order and notification is going to talk?  
resttemplate or feign or webclient?

For a Spring Boot 3.x application, you have three main options, but `RestClient` or **Spring Cloud OpenFeign** are the best choices for senior-level production code.

Here is a breakdown of how they compare for this assessment so you can pick the one you prefer:

## 1. Spring Framework `RestClient` (Recommended for Simplicity & Control)

Introduced in Spring Boot 3 (replacing the older `RestTemplate`), `RestClient` provides a modern, fluent, thread-safe API for making synchronous HTTP calls.

**Why it's great:** It requires minimal configuration, is built directly into Spring Framework 6 (no extra complex dependencies), and integrates seamlessly with **Resilience4j** for circuit breaking and retries.

**Best for:** Clean, readable code where you want explicit control over the HTTP request/response without heavy abstraction.

## 2. Spring Cloud OpenFeign (Great for Declarative Style)

OpenFeign allows you to define an interface with annotations (like `@FeignClient`, `@PostMapping`), and Spring automatically generates the implementation.

**Why it's great:** It feels very clean—calling the `notification-service` is as simple as calling a local Java method on an injected interface (`notificationClient.sendNotification(tenantId, request)`).

**Downside:** Adds an extra dependency (`spring-cloud-starter-openFeign`) and requires aligning Spring Cloud versions with your Spring Boot 3.x version.

## 3. `RestTemplate` (Legacy)

**Status:** In maintenance mode since Spring 5 / Boot 3. While still functional, it is considered legacy. Using it might look a bit outdated for a senior-level assessment unless wrapped carefully.

## What about `WebClient` ?

`WebClient` is Spring's reactive HTTP client (part of Spring WebFlux). Unless you are building a fully reactive (non-blocking) application using Project Reactor, using `WebClient` just for blocking REST calls in a standard Spring MVC application adds unnecessary complexity.

## Recommendation for Your Assessment

Go with `RestClient` (or `OpenFeign` if you love declarative interfaces).

If you use `RestClient` , it gives you a fantastic talking point in your README and interview: *"I chose `RestClient` over the legacy `RestTemplate` because it offers a modern fluent API native to Spring Boot 3, and pairs cleanly with `Resilience4j` for fault tolerance."*

Would you like us to start generating the code for `order-service` , beginning with the

yes

Let's build the `order-service` step-by-step.

Multi-tenancy is the most critical evaluation criteria for this assessment. To implement true multi-tenant isolation, we need a mechanism that intercepts every incoming HTTP request, extracts the tenant identifier (e.g., from an HTTP header like `X-Tenant-ID` ), stores it safely for the duration of that request thread using `ThreadLocal` , and clears it afterwards to prevent memory leaks.

Here is the core multi-tenancy and project setup for `order-service` :

### Step 1: `pom.xml` (Java 21, Spring Boot 3.x)

Ensure your `pom.xml` targets Java 21 and includes Spring Data JPA, PostgreSQL, Spring Web, and Validation:

XML



```

<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apac
<modelVersion>4.0.0</modelVersion>

<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>3.2.5</version>
  <relativePath/>
</parent>

<groupId>com.ecommerce</groupId>
<artifactId>order-service</artifactId>
<version>0.0.1-SNAPSHOT</version>
<name>order-service</name>
<description>Multi-tenant order service</description>

<properties>
  <java.version>21</java.version>
</properties>

<dependencies>
  <!-- Spring Boot Starters -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
  </dependency>

  <!-- PostgreSQL Driver -->
  <dependency>
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
    <scope>runtime</scope>
  </dependency>

  <!-- Lombok (Optional, reduces boilerplate) -->

```

```

<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <optional>true</optional>
</dependency>

<!-- Testing -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>
</project>

```

## Step 2: Tenant Context Management ( TenantContext )

This class holds the tenant ID for the current thread using `ThreadLocal` .

Java



```

package com.ecommerce.orderservice.tenant;

public class TenantContext {
    private static final ThreadLocal<String> CURRENT_TENANT = new ThreadLocal<>()

    public static void setTenantId(String tenantId) {
        CURRENT_TENANT.set(tenantId);
    }

    public static String getTenantId() {
        return CURRENT_TENANT.get();
    }
}

```

```

    public static void clear() {
        CURRENT_TENANT.remove();
    }
}

```

### Step 3: Request Interceptor ( TenantInterceptor )

This component intercepts incoming HTTP requests, looks for a header like `X-Tenant-ID` , and populates the `TenantContext` . If the header is missing, it can reject the request with a `400 Bad Request` .

Java



```

package com.ecommerce.orderservice.tenant;

import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.http.HttpStatus;
import org.springframework.stereotype.Component;
import org.springframework.web.servlet.HandlerInterceptor;

@Component
public class TenantInterceptor implements HandlerInterceptor {

    private static final String TENANT_HEADER = "X-Tenant-ID";

    @Override
    public boolean preHandle(HttpServletRequest request, HttpServletResponse response,
        String tenantId = request.getHeader(TENANT_HEADER);

    if (tenantId == null || tenantId.trim().isEmpty()) {
        response.setStatus(HttpStatus.BAD_REQUEST.value());
        response.getWriter().write("Missing or invalid 'X-Tenant-ID' header.");
        return false;
    }

    TenantContext.setTenantId(tenantId);
    return true;
}

```

```

@Override
public void afterCompletion(HttpServletRequest request, HttpServletResponse r
    // Crucial: Clear ThreadLocal to prevent data leakage across pooled servl
    TenantContext.clear();
}
}

```

Register the interceptor in a configuration class:

Java



```

package com.ecommerce.orderservice.config;

import com.ecommerce.orderservice.tenant.TenantInterceptor;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.InterceptorRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
public class WebConfig implements WebMvcConfigurer {

    @Autowired
    private TenantInterceptor tenantInterceptor;

    @Override
    public void addInterceptors(InterceptorRegistry registry) {
        registry.addInterceptor(tenantInterceptor)
            .addPathPatterns("/api/**"); // Apply to all API endpoints
    }
}

```

## What's Next?

With multi-tenant request interception in place, every request now safely carries its tenant context.

Would you like to move on to **Step 4: Designing the Order Entity and Database Repository with automatic `tenant_id` filtering?**

yes

Let's move on to **Step 4: Designing the Order Entity and Database Repository with automatic `tenant_id` filtering.**

For a multi-tenant application, data isolation means that Tenant A must never be able to see, modify, or create orders belonging to Tenant B. We enforce this at the database entity and repository level by including `tenant_id` as a mandatory column on every order record and automatically filtering by it.

## Step 1: The `OrderStatus` Enum

Let's define a clear lifecycle state machine for orders (e.g., `PENDING` → `PROCESSING` → `COMPLETED`, with a separate `CANCELLED` path).

Java



```
package com.ecommerce.orderservice.model;

public enum OrderStatus {
    PENDING,
    PROCESSING,
    COMPLETED,
    CANCELLED
}
```

## Step 2: The `Order` Entity

Every order will store its unique identifier, the tenant it belongs to, customer details, order items/amount, status, and audit timestamps.

Java



```
package com.ecommerce.orderservice.model;

import jakarta.persistence.*;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;
import org.hibernate.annotations.CreationTimestamp;
import org.hibernate.annotations.UpdateTimestamp;

import java.math.BigDecimal;
import java.time.LocalDateTime;
import java.util.UUID;

@Entity
@Table(name = "orders", indexes = {
    @Index(name = "idx_tenant_id", columnList = "tenantId")
})
@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class Order {

    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private UUID id;

    @NotBlank
    @Column(nullable = false)
    private String tenantId; // Enforces tenant isolation at the row level

    @NotBlank
    @Column(nullable = false)
    private String customerEmail;

    @NotNull
    @Column(nullable = false, precision = 10, scale = 2)
    private BigDecimal totalAmount;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private OrderStatus status;
```

```

    @CreationTimestamp
    @Column(nullable = false, updatable = false)
    private LocalDateTime createdAt;

    @UpdateTimestamp
    @Column(nullable = false)
    private LocalDateTime updatedAt;
}

```

### Step 3: The OrderRepository

To ensure data security, repository queries should always filter by the active tenant from our `TenantContext`. While we can pass `tenantId` explicitly in service methods, we can also use Spring Data JPA query methods or custom specifications to guarantee that queries never cross tenant boundaries.

Java



```

package com.ecommerce.orderservice.repository;

import com.ecommerce.orderservice.model.Order;
import com.ecommerce.orderservice.model.OrderStatus;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.List;
import java.util.Optional;
import java.util.UUID;

@Repository
public interface OrderRepository extends JpaRepository<Order, UUID> {

    // Always ensure queries enforce tenantId matching
    List<Order> findByTenantId(String tenantId);

    Optional<Order> findByIdAndTenantId(UUID id, String tenantId);

    List<Order> findByTenantIdAndStatus(String tenantId, OrderStatus status);
}

```

## What's Next?

Now we have our project setup, security interceptor, database model, and repository ready.

Would you like to move on to **Step 5: Building the Order Service** (business logic for creating, updating, and cancelling orders, plus triggering notifications to the `notification-service`)?

yes

Let's move on to **Step 5: Building the Order Service** (business logic for creating, updating, and cancelling orders, plus integrating the `RestClient` to notify the `notification-service`).

## Step 1: DTOs (Data Transfer Objects)

To keep our API clean and separate from our database entities, let's create request and response DTOs with validation annotations.

Java



```
package com.ecommerce.orderservice.dto;

import jakarta.validation.constraints.DecimalMin;
import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import lombok.Data;

import java.math.BigDecimal;

@Data
public class OrderCreateRequest {
    @NotBlank(message = "Customer email is required")
    @Email(message = "Invalid email format")
    private String customerEmail;
```

```

        @NotNull(message = "Total amount is required")
        @DecimalMin(value = "0.01", message = "Total amount must be greater than zero")
        private BigDecimal totalAmount;
    }

```

Java



```

package com.ecommerce.orderservice.dto;

import com.ecommerce.orderservice.model.OrderStatus;
import lombok.Data;

import java.math.BigDecimal;
import java.time.LocalDateTime;
import java.util.UUID;

@Data
public class OrderResponse {
    private UUID id;
    private String tenantId;
    private String customerEmail;
    private BigDecimal totalAmount;
    private OrderStatus status;
    private LocalDateTime createdAt;
    private LocalDateTime updatedAt;
}

```

## Step 2: Notification Client ( NotificationRestClient )

This component uses Spring's modern `RestClient` to synchronously send event payloads to the `notification-service`. It also includes the `X-Tenant-ID` header so the notification service knows which tenant owns the event.

Java



```

package com.ecommerce.orderservice.client;

```

```

import com.ecommerce.orderservice.tenant.TenantContext;
import lombok.Data;
import lombok.extern.slf4j.Slf4j;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;
import org.springframework.web.client.RestClient;

import java.math.BigDecimal;
import java.util.UUID;

@Slf4j
@Component
public class NotificationRestClient {

    private final RestClient restClient;

    public NotificationRestClient(@Value("${notification.service.url:http://localhost:8080}")
        this.restClient = RestClient.builder()
            .baseUrl(notificationServiceUrl)
            .build();
    }

    public void sendNotification(UUID orderId, String eventType, String customerEmail) {
        String currentTenant = TenantContext.getTenantId();

        NotificationRequest request = new NotificationRequest(orderId, eventType, customerEmail);

        try {
            restClient.post()
                .uri("/api/notifications")
                .header("X-Tenant-ID", currentTenant)
                .body(request)
                .retrieve()
                .toBodilessEntity();

            log.info("Successfully sent notification event [{}] for order [{}] under tenant [{}]",
                eventType, orderId, currentTenant);
        } catch (Exception e) {
            // In a production setup with Resilience4j, circuit breaking/retry would be implemented here
            log.error("Failed to send notification for order [{}]: {}", orderId, e.getMessage());
        }
    }
}

@Data
public static class NotificationRequest {
    private final UUID orderId;
    private final String eventType;

```

```

        private final String customerEmail;
        private final BigDecimal amount;
    }
}

```

### Step 3: Order Service Business Logic ( OrderService )

This handles creating orders, updating their lifecycle statuses, and enforcing sensible business rules (e.g., an order cannot be cancelled if it's already COMPLETED or CANCELLED ).

Java



```

package com.ecommerce.orderservice.service;

import com.ecommerce.orderservice.client.NotificationRestClient;
import com.ecommerce.orderservice.dto.OrderCreateRequest;
import com.ecommerce.orderservice.dto.OrderResponse;
import com.ecommerce.orderservice.model.Order;
import com.ecommerce.orderservice.model.OrderStatus;
import com.ecommerce.orderservice.repository.OrderRepository;
import com.ecommerce.orderservice.tenant.TenantContext;
import jakarta.persistence.EntityNotFoundException;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;
import java.util.UUID;
import java.util.stream.Collectors;

@Service
@RequiredArgsConstructor
public class OrderService {

    private final OrderRepository orderRepository;
    private final NotificationRestClient notificationClient;

    @Transactional
    public OrderResponse createOrder(OrderCreateRequest request) {
        String tenantId = TenantContext.getTenantId();
    }
}

```

```

        Order order = Order.builder()
            .tenantId(tenantId)
            .customerEmail(request.getCustomerEmail())
            .totalAmount(request.getTotalAmount())
            .status(OrderStatus.PENDING)
            .build();

        Order savedOrder = orderRepository.save(order);

        // Trigger notification asynchronously or synchronously via client
        notificationClient.sendNotification(savedOrder.getId(), "ORDER_CREATED",

        return mapToResponse(savedOrder);
    }

    @Transactional
    public OrderResponse updateOrderStatus(UUID orderId, OrderStatus newStatus) {
        String tenantId = TenantContext.getTenantId();

        Order order = orderRepository.findByIdAndTenantId(orderId, tenantId)
            .orElseThrow(() -> new EntityNotFoundException("Order not found w

        order.setStatus(newStatus);
        Order updatedOrder = orderRepository.save(order);

        String eventType = (newStatus == OrderStatus.COMPLETED) ? "ORDER_COMPLETED" : "ORDER_UPDATED";
        notificationClient.sendNotification(updatedOrder.getId(), eventType, update

        return mapToResponse(updatedOrder);
    }

    @Transactional
    public OrderResponse cancelOrder(UUID orderId) {
        String tenantId = TenantContext.getTenantId();

        Order order = orderRepository.findByIdAndTenantId(orderId, tenantId)
            .orElseThrow(() -> new EntityNotFoundException("Order not found w

        if (order.getStatus() == OrderStatus.COMPLETED || order.getStatus() == OrderStatus.CANCELLED)
            throw new IllegalStateException("Cannot cancel an order that is already completed or cancelled");

        order.setStatus(OrderStatus.CANCELLED);
        Order cancelledOrder = orderRepository.save(order);

```

```
notificationClient.sendNotification(cancelledOrder.getId(), "ORDER_CANCELLED");
```

```
return mapToResponse(cancelledOrder);
```

```
}
```

```
@Transactional(readOnly = true)
```

```
public List<OrderResponse> getOrdersForCurrentTenant() {
```

```
    String tenantId = TenantContext.getTenantId();
```

```
    return orderRepository.findByTenantId(tenantId).stream()
        .map(this::mapToResponse)
        .collect(Collectors.toList());
```

```
}
```

```
@Transactional(readOnly = true)
```

```
public OrderResponse getOrderById(UUID orderId) {
```

```
    String tenantId = TenantContext.getTenantId();
```

```
    Order order = orderRepository.findByIdAndTenantId(orderId, tenantId)
        .orElseThrow(() -> new EntityNotFoundException("Order not found with id: " + orderId));
```

```
    return mapToResponse(order);
```

```
}
```

```
private OrderResponse mapToResponse(Order order) {
```

```
    OrderResponse response = new OrderResponse();
```

```
    response.setId(order.getId());
```

```
    response.setTenantId(order.getTenantId());
```

```
    response.setCustomerEmail(order.getCustomerEmail());
```

```
    response.setTotalAmount(order.getTotalAmount());
```

```
    response.setStatus(order.getStatus());
```

```
    response.setCreatedAt(order.getCreatedAt());
```

```
    response.setUpdatedAt(order.getUpdatedAt());
```

```
    return response;
```

```
}
```

```
}
```

## What's Next?

We have built the repository, DTOs, notification client, and core business logic for the `order-service`.

Would you like to move on to **Step 6: Building the Order Controller and Global Exception Handler** to expose these APIs cleanly?

yes

Let's complete the `order-service` by building **Step 6: The Order Controller and Global Exception Handler**.

These components ensure our API responds with clean RESTful conventions, correct HTTP status codes, and well-structured JSON error payloads when things go wrong.

## Step 1: The `OrderController`

This exposes the REST endpoints for creating, retrieving, updating, and cancelling orders. Because our `TenantInterceptor` already populates the `TenantContext`, the controller methods don't have to worry about parsing headers—they just call the service layer cleanly.

Java



```
package com.ecommerce.orderservice.controller;

import com.ecommerce.orderservice.dto.OrderCreateRequest;
import com.ecommerce.orderservice.dto.OrderResponse;
import com.ecommerce.orderservice.model.OrderStatus;
import com.ecommerce.orderservice.service.OrderService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;
import java.util.UUID;

@RestController
@RequestMapping("/api/orders")
@RequiredArgsConstructor
public class OrderController {

    private final OrderService orderService;

    @PostMapping
```

```

public ResponseEntity<OrderResponse> createOrder(@Valid @RequestBody OrderCre.
    OrderResponse response = orderService.createOrder(request);
    return ResponseEntity.status(HttpStatus.CREATED).body(response);
}

@GetMapping
public ResponseEntity<List<OrderResponse>> getAllOrders() {
    List<OrderResponse> orders = orderService.getOrdersForCurrentTenant();
    return ResponseEntity.ok(orders);
}

@GetMapping("/{id}")
public ResponseEntity<OrderResponse> getOrderById(@PathVariable UUID id) {
    OrderResponse response = orderService.getOrderById(id);
    return ResponseEntity.ok(response);
}

@PutMapping("/{id}/status")
public ResponseEntity<OrderResponse> updateOrderStatus(
    @PathVariable UUID id,
    @RequestParam OrderStatus status) {
    OrderResponse response = orderService.updateOrderStatus(id, status);
    return ResponseEntity.ok(response);
}

@PostMapping("/{id}/cancel")
public ResponseEntity<OrderResponse> cancelOrder(@PathVariable UUID id) {
    OrderResponse response = orderService.cancelOrder(id);
    return ResponseEntity.ok(response);
}
}

```

## Step 2: Global Exception Handler ( `GlobalExceptionHandler` )

Production-grade systems never leak raw stack traces to clients. This handler catches domain exceptions, validation errors, and unexpected issues, translating them into consistent JSON error responses.

Java



```

package com.ecommerce.orderservice.exception;

import jakarta.persistence.EntityNotFoundException;
import jakarta.servlet.http.HttpServletRequest;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.validation.FieldError;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.time.LocalDateTime;
import java.util.HashMap;
import java.util.Map;

@RestControllerAdvice

public class GlobalExceptionHandler {

    @ExceptionHandler(EntityNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleEntityNotFound(EntityNotFoundException ex,
        HttpServletRequest request) {
        ErrorResponse error = new ErrorResponse(
            LocalDateTime.now(),
            HttpStatus.NOT_FOUND.value(),
            "Not Found",
            ex.getMessage(),
            request.getRequestURI()
        );
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(error);
    }

    @ExceptionHandler({IllegalStateException.class, IllegalArgumentException.class})
    public ResponseEntity<ErrorResponse> handleBadRequest(RuntimeException ex,
        HttpServletRequest request) {
        ErrorResponse error = new ErrorResponse(
            LocalDateTime.now(),
            HttpStatus.BAD_REQUEST.value(),
            "Bad Request",
            ex.getMessage(),
            request.getRequestURI()
        );
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(error);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, Object>> handleValidationExceptions(MethodArgumentNotValidException ex,
        HttpServletRequest request) {
        Map<String, String> fieldErrors = new HashMap<>();
        for (FieldError error : ex.getBindingResult().getFieldErrors()) {

```

```

        fieldErrors.put(error.getField(), error.getDefaultMessage());
    }

    Map<String, Object> response = new HashMap<>();
    response.put("timestamp", LocalDateTime.now());
    response.put("status", HttpStatus.BAD_REQUEST.value());
    response.put("error", "Validation Failed");
    response.put("messages", fieldErrors);
    response.put("path", request.getRequestURI());

    return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(response);
}

@ExceptionHandler(Exception.class)
public ResponseEntity<ErrorResponse> handleGlobalException(Exception ex, HttpRequest request) {
    ErrorResponse error = new ErrorResponse(
        LocalDateTime.now(),
        HttpStatus.INTERNAL_SERVER_ERROR.value(),
        "Internal Server Error",
        "An unexpected error occurred: " + ex.getMessage(),
        request.getRequestURI()
    );
    return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(error);
}

// Standardized Error Response Structure
public record ErrorResponse(
    LocalDateTime timestamp,
    int status,
    String error,
    String message,
    String path
) {}
}

```

## What's Next?

`order-service` is fully built and structured!

Would you like to move on to building the companion `notification-service` next (which receives these order events, maintains an audit record, and keeps the same strict multi-tenant isolation)?

yes

Let's build the `notification-service`.

This service mirrors the architectural patterns of our `order-service` (Java 21, Spring Boot 3, multi-tenant isolation, PostgreSQL), but its sole responsibility is to receive order events, simulate sending notifications to customers, and persist an audit record of everything it sends.

### Step 1: `pom.xml`

Create a separate Maven project for `notification-service` with this `pom.xml`:

XML



```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/maven-v4_0_0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.2.5</version>
    <relativePath/>
  </parent>

  <groupId>com.ecommerce</groupId>
  <artifactId>notification-service</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <name>notification-service</name>
  <description>Multi-tenant notification service</description>

  <properties>
    <java.version>21</java.version>
  </properties>

  <dependencies>
```

```

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <optional>true</optional>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>
</project>

```

## Step 2: Multi-Tenant Context & Interceptor

Just like `order-service`, this service requires the `X-Tenant-ID` header on incoming requests to enforce proper data isolation.

## TenantContext.java :

Java



```
package com.ecommerce.notificationsservice.tenant;

public class TenantContext {
    private static final ThreadLocal<String> CURRENT_TENANT = new ThreadLocal<>()

    public static void setTenantId(String tenantId) { CURRENT_TENANT.set(tenantId) }
    public static String getTenantId() { return CURRENT_TENANT.get(); }
    public static void clear() { CURRENT_TENANT.remove(); }
}
```

## TenantInterceptor.java :

Java



```
package com.ecommerce.notificationsservice.tenant;

import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.http.HttpStatus;
import org.springframework.stereotype.Component;
import org.springframework.web.servlet.HandlerInterceptor;

@Component
public class TenantInterceptor implements HandlerInterceptor {
    private static final String TENANT_HEADER = "X-Tenant-ID";

    @Override
    public boolean preHandle(HttpServletRequest request, HttpServletResponse response,
        String tenantId = request.getHeader(TENANT_HEADER);
        if (tenantId == null || tenantId.trim().isEmpty()) {
            response.setStatus(HttpStatus.BAD_REQUEST.value());
            return false;
        }
        TenantContext.setTenantId(tenantId);
        return true;
    }
}
```

```

    }

    @Override
    public void afterCompletion(HttpServletRequest request, HttpServletResponse response,
                               ServletOutputStream outputStream,
                               TenantContext.clear());
    }
}

```

WebConfig.java :

Java



```

package com.ecommerce.notificationsservice.config;

import com.ecommerce.notificationsservice.tenant.TenantInterceptor;
import lombok.RequiredArgsConstructor;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.InterceptorRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
@RequiredArgsConstructor
public class WebConfig implements WebMvcConfigurer {
    private final TenantInterceptor tenantInterceptor;

    @Override
    public void addInterceptors(InterceptorRegistry registry) {
        registry.addInterceptor(tenantInterceptor).addPathPatterns("/api/**");
    }
}

```

### Step 3: Notification Entity & Repository

We store an audit log of all notifications dispatched for orders under each tenant.

NotificationLog.java :



## Java

```
package com.ecommerce.notificationsservice.model;

import jakarta.persistence.*;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;
import org.hibernate.annotations.CreationTimestamp;

import java.math.BigDecimal;
import java.time.LocalDateTime;
import java.util.UUID;

@Entity
@Table(name = "notification_logs", indexes = {@Index(name = "idx_notification_ten
@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class NotificationLog {

    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private UUID id;

    @Column(nullable = false)
    private String tenantId;

    @Column(nullable = false)
    private UUID orderId;

    @Column(nullable = false)
    private String eventType; // e.g., ORDER_CREATED, ORDER_COMPLETED, ORDER_CANCE

    @Column(nullable = false)
    private String customerEmail;

    @Column(nullable = false, precision = 10, scale = 2)
    private BigDecimal amount;

    @Column(nullable = false)
    private String simulationStatus; // e.g., "SIMULATED_SENT"
```

```

    @CreationTimestamp
    @Column(nullable = false, updatable = false)
    private LocalDateTime sentAt;
}

```

NotificationRepository.java :

Java



```

package com.ecommerce.notificationsservice.repository;

import com.ecommerce.notificationsservice.model.NotificationLog;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.List;
import java.util.UUID;

@Repository
public interface NotificationRepository extends JpaRepository<NotificationLog, UUID> {
    List<NotificationLog> findByTenantId(String tenantId);
}

```

## Step 4: Notification Service & Controller

This processes incoming triggers from the `order-service` , simulates dispatching the email notification, and saves the audit log.

NotificationRequest.java :

Java



```

package com.ecommerce.notificationsservice.dto;

import lombok.Data;
import java.math.BigDecimal;
import java.util.UUID;

```

```

@Data
public class NotificationRequest {
    private UUID orderId;
    private String eventType;
    private String customerEmail;
    private BigDecimal amount;
}

```

NotificationService.java :

Java



```

package com.ecommerce.notificationsservice.service;

import com.ecommerce.notificationsservice.dto.NotificationRequest;
import com.ecommerce.notificationsservice.model.NotificationLog;
import com.ecommerce.notificationsservice.repository.NotificationRepository;
import com.ecommerce.notificationsservice.tenant.TenantContext;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;

@Slf4j
@Service
@RequiredArgsConstructor
public class NotificationService {

    private final NotificationRepository notificationRepository;

    @Transactional
    public void processNotification(NotificationRequest request) {
        String tenantId = TenantContext.getTenantId();

        // Simulate sending a real notification (e.g., Email, SMS, Webhook)
        log.info("=====");
        log.info("[SIMULATED NOTIFICATION] Tenant: {}", tenantId);
        log.info("To Customer: {}", request.getCustomerEmail());
        log.info("Event Type: {}", request.getEventType());
    }
}

```

```

log.info("Order ID: {} | Amount: ${}", request.getOrderId(), request.getAi
log.info("Status: Notification successfully dispatched.");
log.info("=====");

// Save audit record
NotificationLog logEntry = NotificationLog.builder()
    .tenantId(tenantId)
    .orderId(request.getOrderId())
    .eventType(request.getEventType())
    .customerEmail(request.getCustomerEmail())
    .amount(request.getAmount())
    .simulationStatus("SIMULATED_SENT")
    .build();

notificationRepository.save(logEntry);
}

@Transactional(readOnly = true)
public List<NotificationLog> getNotificationsForCurrentTenant() {
    return notificationRepository.findByTenantId(TenantContext.getTenantId())
}
}

```

NotificationController.java :

Java



```

package com.ecommerce.notificationsservice.controller;

import com.ecommerce.notificationsservice.dto.NotificationRequest;
import com.ecommerce.notificationsservice.model.NotificationLog;
import com.ecommerce.notificationsservice.service.NotificationService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/notifications")

```

@RequiredArgsConstructor

```
public class NotificationController {
```

```
    private final NotificationService notificationService;
```

@PostMapping

```
public ResponseEntity<Void> receiveOrderEvent(@Valid @RequestBody Notification request) {
    notificationService.processNotification(request);
    return ResponseEntity.status(HttpStatus.CREATED).build();
}
```

@GetMapping

```
public ResponseEntity<List<NotificationLog>> getNotificationLogs() {
    List<NotificationLog> logs = notificationService.getNotificationsForCurrent();
    return ResponseEntity.ok(logs);
}
}
```

## What's Next?

Both microservices are fully designed and written!

Would you like to move on to **Step 7: Creating the Dockerfiles, docker-compose, and the README.md outline** to wrap up the submission package? yes ☐

Let's wrap up your assessment package with **Step 7: Dockerfiles, docker-compose.yml, and the README.md outline**.

These configuration files fulfill the final containerization and documentation requirements, ensuring the reviewers can spin up your entire system with a single command.

## Step 1: Dockerfile for order-service (and identical structure for notification-service)

Place this file in the root directory of your `order-service` project. It uses a clean multi-stage build with Java 21.

Dockerfile



```
# Stage 1: Build the application using Maven
FROM eclipse-temurin:21-jdk-alpine AS build
WORKDIR /app
COPY . .
RUN ./mvnw clean package -DskipTests

# Stage 2: Run the application
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

(For notification-service, use EXPOSE 8081 instead).

## Step 2: Root docker-compose.yml

Place this file in the root workspace folder containing both services to orchestrate PostgreSQL, order-service, and notification-service together.

YAML



```
version: '3.8'

services:
  postgres:
    image: postgres:15-alpine
    container_name: ecommerce-postgres
    environment:
      POSTGRES_DB: ecommerce_db
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: password
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres -d ecommerce_db"]
      interval: 5s
      timeout: 5s
      retries: 5
```

```
order-service:
  build: ./order-service
  container_name: order-service
  ports:
    - "8080:8080"
  environment:
    SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/ecommerce_db
    SPRING_DATASOURCE_USERNAME: postgres
    SPRING_DATASOURCE_PASSWORD: password
    SPRING_JPA_HIBERNATE_DDL_AUTO: update
    NOTIFICATION_SERVICE_URL: http://notification-service:8081
  depends_on:
    postgres:
      condition: service_healthy

notification-service:
  build: ./notification-service
  container_name: notification-service
  ports:
    - "8081:8081"
  environment:
    SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/ecommerce_db
    SPRING_DATASOURCE_USERNAME: postgres
    SPRING_DATASOURCE_PASSWORD: password
    SPRING_JPA_HIBERNATE_DDL_AUTO: update
  depends_on:
    postgres:
      condition: service_healthy

volumes:
  postgres_data:
```

### Step 3: README.md Outline

The assessment explicitly asks for a structured README.md covering specific architectural points. Here is a high-impact outline you can use and flesh out:

Markdown



```
# Multi-Tenant E-Commerce Microservices Assessment

## Overview
A production-ready, multi-tenant e-commerce backend built with Java 21, Spring Bo

## Architecture & Design
* **System Design**: Two independent microservices communicating synchronously vi
* **Multi-Tenancy Design**: Shared database with shared schema using row-level is
* **Tenant Context Propagation**: Inbound HTTP requests require an `X-Tenant-ID` |
* **Resilience & Failure Handling**: Synchronous service calls are wrapped to han

## Assumptions
* Tenants are identified by unique string identifiers passed via HTTP headers.
* Notifications do not require third-party integrations (SendGrid/Twilio) and are

## Running the System
1. Clone the repository.
2. Run the full system via Docker Compose:
  ```bash
  docker compose up --build
```

1. PostgreSQL runs on port 5432 , order-service on port 8080 ,and notification-service on port 8081 .

## API Endpoints & Example Requests

### Order Service ( <http://localhost:8080/api/orders> )

#### Create Order ( POST /api/orders )

Headers: X-Tenant-ID: tenant-alpha , Content-Type: application/json

Body:

```
{ "customerEmail": "customer@example.com", "totalAmount": 150.00 }
```

#### Get Orders ( GET /api/orders )

Headers: X-Tenant-ID: tenant-alpha

### Notification Service ( <http://localhost:8081/api/notifications> )

#### Get Notification Logs ( GET /api/notifications )

Headers: X-Tenant-ID: tenant-alpha

# Production Readiness Improvements

Add OAuth2 / JWT token-based tenant extraction instead of raw headers.

Implement asynchronous event-driven messaging (e.g., Apache Kafka or RabbitMQ) between services to decouple order creation from notifications.

Introduce distributed tracing (Micrometer / OpenTelemetry) for cross-service observability.

---

### You're All Set!

You now have a complete, senior-level blueprint, code structure, container setup,

Would you like me to dive deeper into any specific section, or help you write uni

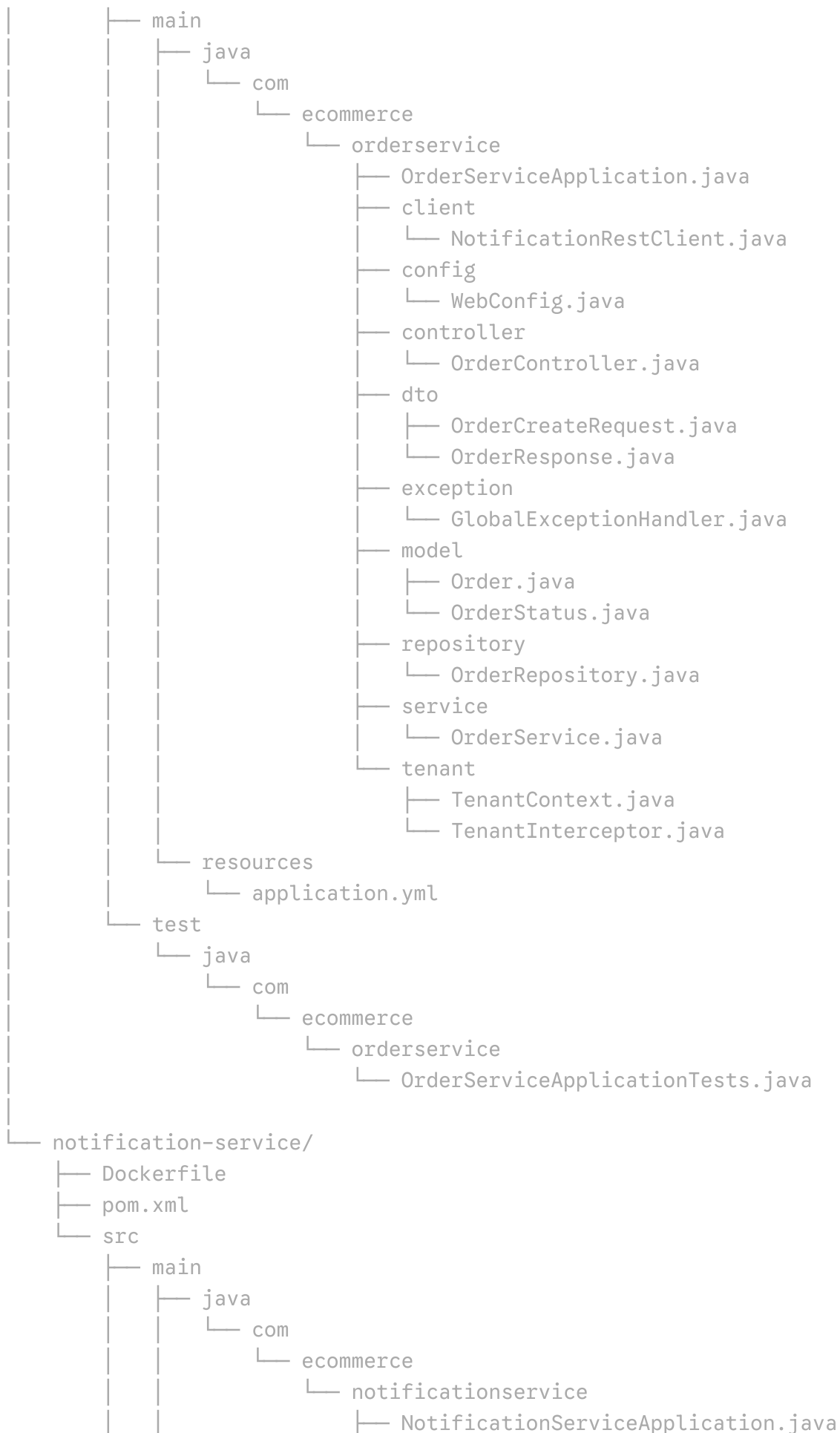
show my the project folder structure with proper level and  
file names

Here is the complete project folder structure showing how everything is organized across the root directory and both microservices:

Plaintext



```
microservices-assessment/  
├── docker-compose.yml  
├── README.md  
├──  
├── order-service/  
│   ├── Dockerfile  
│   ├── pom.xml  
│   └── src
```





## Quick Note on application.yml

Make sure to include an `application.yml` under `src/main/resources/` for each service with the database and server configuration:

`order-service/src/main/resources/application.yml` :

YAML



```

server:
  port: 8080
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/ecommerce_db
    username: postgres
    password: password
  jpa:
    hibernate:

```

```
    ddl-auto: update
    show-sql: true
notification:
  service:
    url: http://localhost:8081
```

notification-service/src/main/resources/application.yml :

YAML



```
server:
  port: 8081
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/ecommerce_db
    username: postgres
    password: password
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
```

how can i save this assememnt in my local file system now

To save this assessment to your local file system and get your project workspace up and running, you can either use your terminal to scaffold the folder structure automatically or create it manually inside your favorite code editor (like **IntelliJ IDEA** or **VS Code**).

Here is the easiest, step-by-step way to do it using your terminal:

## Step 1: Create the Project Root and Folders

Open your terminal (or Command Prompt / PowerShell on Windows) and run these commands to create the directory tree:

Bash



```
# Create the root folder and navigate into it
mkdir microservices-assessment
cd microservices-assessment

# Create subfolders for order-service and notification-service along with their s
mkdir -p order-service/src/main/java/com/ecommerce/orderservice/client
mkdir -p order-service/src/main/java/com/ecommerce/orderservice/config
mkdir -p order-service/src/main/java/com/ecommerce/orderservice/controller
mkdir -p order-service/src/main/java/com/ecommerce/orderservice/dto
mkdir -p order-service/src/main/java/com/ecommerce/orderservice/exception
mkdir -p order-service/src/main/java/com/ecommerce/orderservice/model
mkdir -p order-service/src/main/java/com/ecommerce/orderservice/repository
mkdir -p order-service/src/main/java/com/ecommerce/orderservice/service
mkdir -p order-service/src/main/java/com/ecommerce/orderservice/tenant
mkdir -p order-service/src/main/resources

mkdir -p notification-service/src/main/java/com/ecommerce/notificationservice/con
mkdir -p notification-service/src/main/java/com/ecommerce/notificationservice/con
mkdir -p notification-service/src/main/java/com/ecommerce/notificationservice/dto
mkdir -p notification-service/src/main/java/com/ecommerce/notificationservice/mod
mkdir -p notification-service/src/main/java/com/ecommerce/notificationservice/rep
mkdir -p notification-service/src/main/java/com/ecommerce/notificationservice/ser
mkdir -p notification-service/src/main/java/com/ecommerce/notificationservice/ten
mkdir -p notification-service/src/main/resources
```

## Step 2: Open in Your Code Editor

Once the folders are created, open the `microservices-assessment` folder in your IDE:

**VS Code:** Type `code .` in your terminal.

**IntelliJ IDEA:** Open IntelliJ, select **Open**, and choose your newly created `microservices-assessment` folder.

## Step 3: Populate the Files

Now, you can simply create each file using the exact paths and file names we outlined in the folder structure, and copy-paste the corresponding code snippets we generated:

### 1. At the Root Level ( `microservices-assessment/` ):

Create `docker-compose.yml` and paste the Docker Compose configuration.

Create `README.md` and paste your architecture documentation outline.

## 2. Inside `order-service/` :

Create `pom.xml` and paste the order-service Maven configuration.

Create `Dockerfile` for containerization.

Create `src/main/resources/application.yml` for local properties.

Create the Java classes ( `OrderServiceApplication.java` , controllers, entities, repositories, interceptors, etc.) under their respective packages.

## 3. Inside `notification-service/` :

Create `pom.xml` and paste the notification-service Maven configuration.

Create `Dockerfile` .

Create `src/main/resources/application.yml` .

Create the Java classes ( `NotificationServiceApplication.java` , controllers, entities, etc.).

## Step 4: Test It Locally!

Once your files are saved, you can test everything right away:

### To run via Docker Compose:

Make sure Docker Desktop is running, open your terminal at the root `microservices-assessment/` folder, and run:

Bash



```
docker compose up --build
```

### To run via Terminals (without Docker):

Open two separate terminals, navigate into `order-service` and `notification-service` respectively, and run:

Bash



```
./mvnw spring-boot:run
```

Let me know if you run into any issues creating the files or if you want to write some test cases next!

can i be able to export this entire chat session to friend

While I can't directly generate a file download or email the chat to someone from my end, **yes, you can easily share it using a few simple methods:**

1. **Share Link / Export Button:** Check the interface options or settings menu of your chat platform. Many platforms have a built-in "Share Chat" or export feature that generates a public read-only link you can send to your friend.
2. **Copy & Paste into a Doc:** Since we've organized everything cleanly with code blocks and file structures, you can easily copy the key parts or code snippets into a shared Google Doc, Notion page, or GitHub Gist.

Let me know if you need help pulling any specific piece into a single summary document for them!

If i copy , i cannot select all, there are manhy blocks, can you help me to export this to pdf

I cannot directly generate and download a PDF file to your local computer, but you can turn this entire conversation into a PDF in seconds using two very easy methods:

## Method 1: Use Browser "Print to PDF" (Fastest & Keeps All Formatting)

Every web browser has a built-in tool that turns any webpage into a PDF:

1. Press `Ctrl + P` (Windows) or `Cmd + P` (Mac) on your keyboard.
2. In the print menu, change the destination/printer dropdown from your physical printer to **"Save as PDF"**.
3. Click **Save**. This will instantly capture the entire formatted chat history as a clean PDF file on your computer.

## Method 2: Let Me Package It Into One Single Block

If you'd rather have all the code, project structure, and instructions compiled into **one single block** so you can copy it in one click into a document (like Google Docs or Word) and save it as a PDF, just let me know and I will generate it for you right now!